```
pylookup = ChainMap(locals(), globals(), vars(builtins)
```

MappingProxyType builds a read-only dictionary instance.

Performance

Time complexity for querying is O(1), while time complexity for storing is O(n). Calculating the count is going to slow down a dictionary; it will take O(n) time complexity to calculate the counts for all keys (because each key will have to be hit at least once to append its count to the running count stored in the value).

Space complexity for storing is O(n). (During hash collision, it is more than O(n).)

Let's look at a sample to see the performance list versus dict. Here's the code:

```
import timeit
def find_number_in_list(lst, number):
   if number in lst:
       return True
   else:
       return False
short_list = list(range(100))
start = timeit.default_timer()
find_number_in_list(short_list, 1)
stop = timeit.default_timer()
print("--- short_list %s seconds ---" % (stop - start))
long_list = list(range(10000000))
start = timeit.default_timer()
find_number_in_list(long_list, 1)
stop = timeit.default_timer()
print("--- long_list %s seconds ---" % (stop - start))
def find_number_in_dict(dct, number):
   if number in dct.keys():
       return True
   else:
       return False
short_dict = {x:x*5 for x in range(1,100)}
start = timeit.default_timer()
find_number_in_dict(short_dict, 1)
stop = timeit.default_timer()
print("--- short_dict %s seconds ---" % (stop - start))
long_dict = {x:x*5 for x in range(1,10000000)}
find_number_in_dict(long_dict, 1)
stop = timeit.default_timer()
print("--- long_dict %s seconds ---" % (stop - start))
```

The output is:

```
>> python list_vs_dict_time.py
--- short_list 1.0299999999990872e-06 seconds ---
--- long_list 4.7639999999904425e-06 seconds ---
--- short_dict 2.97399999998893e-06 seconds ---
--- long_dict 1.768262919 seconds ---
```

The fastest way to repeatedly lookup data with millions of entries in Python is by using dictionaries. Because dictionaries have built-in mapping in Python, they are highly optimised. However, there is a typical space-time trade-off in dictionaries and lists. Dict still uses more memory than lists, since you need to use space for the keys and the lookup as well, while lists use space only for the values.

In general, O(1) is used to find the element in the dictionary. Though when this is a hash collision, it's not always O(1).

Dictionaries are a preferable option for mapping key-value elements, as they offer far better performance compared to lists and tuples, especially when it comes to search, add, and delete operations. Sets are similar to dictionaries, but they lack key-value pairing and consist of unique, disordered elements. We will learn more about sets and the underlying concept of hash in the next article in the series. END

References

- *https://nolongerset.com/content/images/2020/12/key-5105878_1920.jpg*
- Fluent Python - Luiano Ramalho
- *https://github.com/python/cpython/blob/main/Objects/dictobject.c*
- *https://stackoverflow.com/questions/52023758/list-vs-dictionary-vs-set-for-finding-the-number-of-times-a-number-appears*
- *https://towardsdatascience.com/faster-lookups-in-python-1d7503e9cd38*
- *https://dzone.com/articles/python-memo-2-dictionary-vs-set-1*
- *https://www.oreilly.com/library/view/high-performance-python/9781449361747/ch04.html*
- *https://tenthousandmeters.com/blog/python-behind-the-scenes-10-how-python-dictionaries-work/*
- *https://realpython.com/python-hash-table/*
- *https://www.acunetix.com/vulnerabilities/web/php-hash-collision-denial-of-service-vulnerability/*
- *https://www.asmeurer.com/blog/posts/what-happens-when-you-mess-with-hashing-in-python/*
- *https://www.youtube.com/watch?v=npw4s1QTmPg*
- *https://www.geeksforgeeks.org/*/*
- *https://morepypy.blogspot.com/2015/01/faster-more-memory-efficient-and-more.html*
- *https://mail.python.org/pipermail/python-dev/2012-December/123028.html*
- *https://www.fluentpython.com/extra/internals-of-sets-and-dicts/*
- *https://www.youtube.com/watch?v=66P5FMkWoVU&t=56s*

By: Thangaselvi Arichandrapandian

The author is an all-time learner influenced by the quote: *"The more I learn, the more I realise how much I don't know." -Albert Einstein*